简介

虚拟机（Virtual Machine，VM）是一种由软件实现的计算机执行环境。
它并非简单地“模拟硬件”，而是**人为定义了一套独立于真实 CPU 的执行模型**：包括指令集、寄存器/栈结构、内存访问规则以及控制流语义。代码运行在这套模型之上，而非直接运行在物理处理器之上。

从工程角度看，虚拟机可以屏蔽底层硬件与平台差异；
从分析与对抗角度看，虚拟机更重要的价值在于：**执行语义完全可控、可重写、可混淆**。
这也是虚拟机保护（VMP）能够成立的根本原因。

常见的虚拟机包括 JVM（Java Virtual Machine）、Python VM、Dart VM 等。这些系统功能完整、历史包袱沉重，源码规模庞大，直接对其进行整体分析，对初学者而言成本极高。
相比之下，Lua 虚拟机结构紧凑、解释器路径清晰、代码风格克制，是理解“虚拟机究竟在做什么”的理想起点。

我学习虚拟机的直接动机来自 VMP。
在实际逆向中，基于 trace、unidbg、RPC 等手段的方案，本质上仍然是
在虚拟机之外绕行，而非直面虚拟机本身。当代码的真实语义被迁移到一套人为设计的执行模型中时，分析的难点不再是某一条指令或某一个函数，而是**整个执行语义的重建**。

因此，本系列将从一个语言级虚拟机入手，围绕其解释器实现，理解以下问题：

- 虚拟机的执行模型是什么；
- 为什么虚拟机在分析上天然更困难；
- 在面对虚拟机代码时，分析者需要额外构造哪些“语义工具”。

环境搭建

本系列选用 Lua 5.5 作为研究对象，实验环境为 Linux。

可使用物理机、虚拟机或 WSL 进行搭建。

```
curl -L -R -O https://www.lua.org/ftp/lua-5.5.0.tar.gz
tar zxf lua-5.5.0.tar.gz
cd lua-5.5.0
make all test
```

以上命令将完成 Lua 源码的编译与基础测试。

生成编译数据库（推荐）

为便于后续使用 clangd、ccls 等代码分析工具进行源码阅读，建议生成编译数据库：

```
bear -- make all test
```

执行完成后，将生成 `compile_commands.json` 文件，可用于精确跳转函数定义、追踪调用关系以及辅助静态分析。

编译器说明

后续源码分析过程中，将同时使用 GCC 与 Clang 进行编译与对比。

两者在代码生成策略、优化行为以及调试友好性方面均存在差异，这些差异会直接影响我们对虚拟机执行行为的观察。相关内容将在分析解释器实现时展开说明。

结语

这不是一篇试图“讲完虚拟机”的文章，而只是一个起点。

后续内容将直接进入 Lua 虚拟机的解释器实现，从 opcode 分发、执行循环与运行时状态入手，逐步拆解其真实的执行语义。

当我们能够读懂一个小型语言虚拟机在二进制层面**如何执行代码**时，再回头看 VMP，其难点与本质将不再抽象。